

Aprendizagem Baseada em Problemas (PBL): Comunicação em Sistemas Distribuídos

Uma Abordagem Popperiana

Faculdade de Computação e Informática – FCI

Universidade Presbiteriana Mackenzie

A transição de sistemas monolíticos para sistemas distribuídos exige uma ruptura de paradigmas. Sob a ótica de Karl Popper, o nosso conhecimento avança quando nossas expectativas teóricas falham diante da realidade. Na comunicação em sistemas distribuídos, a falha (latência, perda de pacotes, crash do servidor) é a regra, não a exceção.

Os problemas a seguir foram desenhados com base no material de referência (RPC, Web Services, gRPC, Sockets e Message-Oriented Middleware) para forçar o aluno a abandonar a ilusão de que invocar um método remoto é o mesmo que invocar um método local. Através de cenários práticos aplicados a domínios críticos (Saúde, Nuclear e HPC), os estudantes deverão investigar, formular conjecturas e propor arquiteturas resilientes.

Os Problemas (PBL)

Problema 1: A Ilusão da Chamada Local (Nível: Introdutório)

Cenário: Durante a preparação de uma atividade prática para a disciplina de *Extração de Conhecimento*, um aluno implementou um sistema Cliente/Servidor básico utilizando RPC síncrono padrão. O objetivo do sistema é consultar a base de dados central do instituto para validar o cadastro de pesquisadores. No laboratório (rede local), o sistema funcionou perfeitamente. No entanto, ao implantar o servidor em uma máquina virtual externa e distribuir o cliente para que os alunos testassem de suas casas, a aplicação cliente começou a “congelar” a interface por vários segundos a cada clique e, em momentos de pico, o servidor rejeitava conexões.

O Desafio Popperiano: A teoria clássica de procedimentos (black box) assumida pelo aluno ruiu diante do ambiente de rede amplo.

Tarefa de Investigação:

1. Diagnostique a causa raiz do congelamento da interface no cliente e da sobrecarga no servidor, referenciando especificamente os “Tipos de Comunicação” (transiente/persistente, síncrono/assíncrono).

Problema 1: A Ilusão da Chamada Local (Nível: Introdutório) (cont.)

2. Como a semântica de passagem de parâmetros e o papel dos *stubs* podem ser modificados para evitar o bloqueio da aplicação cliente? Proponha uma alteração arquitetural usando RPCs Assíncronos.

Problema 2: O Gargalo Logístico e o Peso do XML (Nível: Intermediário)

Cenário: Um projeto de pesquisa focado na otimização da localização de plantas de manufatura de radiofármacos no Brasil precisa de um sistema de telemetria em tempo real. Cada veículo de transporte emite a cada 500 milissegundos um pacote de dados contendo coordenadas GPS, temperatura e níveis de radiação do contêiner.

Atualmente, a equipe tentou prototipar a comunicação usando um Web Service baseado em REST (sobre HTTP/1.1) enviando pacotes JSON, mas o consumo de banda na rede móvel (3G/4G) está insustentável, e a sobrecarga de abrir e fechar conexões HTTP para cada requisição está esgotando os recursos do servidor de recepção.

O Desafio Popperiano: A crença de que REST é a bala de prata para qualquer API moderna é refutada pelas restrições de latência, volume e estado de conexão da telemetria de alta frequência.

Tarefa de Investigação:

Problema 2: O Gargalo Logístico e o Peso do XML (Nível: Intermediário) (cont.)

1. Analise o modelo REST atual e explique por que a natureza *stateless* e o uso de HTTP/1.1 padrão se tornam gargalos críticos neste cenário específico de *streaming* contínuo.
2. Projete uma nova interface de comunicação utilizando **gRPC**. Justifique como o uso de *Protocol Buffers* (em vez de JSON) e a multiplexação do HTTP/2 resolvem os problemas de banda e conexão. Apresente um rascunho de como seria o arquivo `.proto` para o envio contínuo (*client streaming*) dos dados de telemetria.

Problema 3: Orquestração de Imagens no CINAS (Nível: Avançado)

Cenário: O Grupo de Computação e Inteligência Artificial (CINAS) possui um *cluster* de desktops físicos rodando Kubuntu, utilizados para processamento pesado de imagens médicas em algoritmos de Machine Learning. A comunicação atual falha constantemente porque o nó central tenta distribuir tarefas via conexões TCP diretas (*Sockets* clássicos), mas os *workers* (desktops) frequentemente ficam indisponíveis devido a reinicializações ou quedas na rede. Além disso, a equipe de pesquisadores precisa de um *dashboard* centralizado que mostre em tempo real qual nó está processando qual lote, sem interferir no fluxo de trabalho principal.

O Desafio Popperiano: A comunicação transiente, que exige que emissor e receptor estejam ativos simultaneamente, é inadequada para um ambiente computacional distribuído sujeito a falhas de hardware e rede intermitentes.

Tarefa de Investigação:

Desenvolva uma arquitetura híbrida de *Message-Oriented Middleware* para resolver esse problema.

Problema 3: Orquestração de Imagens no CINAS (Nível: Avançado) (cont.)

1. Especifique qual padrão de enfileiramento (ex: *Pipeline*) deve ser usado para distribuir a carga de trabalho (*workload*) de processamento de imagens do Nó Central para os Desktops.
2. Determine se a comunicação para distribuição das tarefas deve ser Transiente (ex: ZeroMQ) ou Persistente via Message Broker (ex: AMQP/RabbitMQ). Justifique sua escolha considerando tolerância a falhas.
3. Para o *dashboard* de monitoramento, sugira a implementação do padrão *Publish-Subscribe*. Como ele se integraria à arquitetura principal?

Referências e Ferramentas Sugeridas para os Alunos

- **Fundamentos:** Tanenbaum, A. S., & Van Steen, M. (2017). *Distributed Systems: Principles and Paradigms* (3rd ed.). (Caps. 4 - Communication).
- **Implementação gRPC:** Subramanian, S. (2019). *gRPC: Up and Running*. O'Reilly Media. (Cap. 2 sobre Protocol Buffers e Cap. 3 sobre Padrões de Comunicação).
- **Filas e Brokers:** Documentação oficial do RabbitMQ (Protocolo AMQP) e tutoriais da arquitetura ZeroMQ (The Guide - zguide.zeromq.org).
- **Material de Aula:** Slides de Computação Distribuída - FCI/Mackenzie (Foco em *Sockets*, RPC, gRPC e Middleware Orientado a Mensagens).